

1. Estructuras y definiciones de hardware

Más allá de los accesos a memoria, para gestionar todos los elementos mapeados en memoria del procesador y periféricos, existen ciertas construcciones en C que permiten facilitar el trabajo al programador de diferentes maneras. Entre ellas, podremos evitar optimizaciones no deseadas con la palabra clave `volatile`, generar `struct` que representen periféricos, acceder a nivel de bits con los campos de bits, u optimizar llamadas a funciones `static inline`.

1.1. La palabra clave `volatile`

El optimizador del compilador asume un entorno de ejecución mono-hilo clásico, donde los valores almacenados en memoria solo cambian mediante instrucciones explícitas del propio código. Bajo esta premisa, si el compilador detecta lecturas repetidas a una misma dirección de memoria dentro de un bucle, optimiza el acceso cargando dicho valor en un registro interno del procesador y reutilizándolo, evitando lecturas redundantes al bus de datos.

En sistemas empotrados, esta optimización no es correcta: los registros mapeados en memoria, o las variables compartidas con Rutinas de Servicio de Interrupción (ISR) cambian de valor de forma asíncrona debido al hardware externo.

```
1 // Direccion de un registro de estado del hardware (por ejemplo, UART RX Ready)
2 #define UART_STATUS_REG (*(uint32_t *) 0x40001000U)
3
4 void esperar_datos(void) {
5     // El compilador lee el registro UNA SOLA VEZ y genera un bucle infinito
6     // en ensamblador si el valor inicial era 0: "if (!reg) while(1);"
7     while ((UART_STATUS_REG & 0x01U) == 0) {
8         // Bucle de espera activa (Busy-wait)
9     }
10 }
```

Al calificar una variable o puntero con `volatile`, se le indica explícitamente al compilador que no optimice sus accesos y lo lea explícitamente cada vez que el código lo requiera. Así, cada lectura o escritura escrita en C se traduce explícitamente en una instrucción física de lectura o escritura en la memoria.

```
1 // Puntero a registro hardware: siempre calificado como volatile
2 #define UART_STATUS_REG (*(volatile uint32_t *) 0x40001000U)
3
4 // Variable global modificada dentro de una ISR
5 volatile uint8_t flag_interrupcion = 0;
```

1.2. Abstracción de Hardware con `struct`

Escribir macros o punteros independientes para cada uno de los decenas de registros que contiene un periférico sería, francamente, un rollo. Puesto que los registros de un periférico ocupan bloques contiguos de memoria, es habitual agruparlos dentro de un (`struct`) de C.

Al definir la estructura, el compilador asigna a cada miembro direcciones consecutivas de memoria desde la inicial. Bastará por tanto con configurar adecuadamente esta dirección inicial del `struct`, con la que especifiquen los manuales del procesador.

```

1  #include <stdint.h>
2
3  typedef struct {
4      volatile uint32_t MODER;    // Offset: 0x00 (Modo de los pines)
5      volatile uint32_t OTYPER;   // Offset: 0x04 (Tipo de salida)
6      volatile uint32_t OSPEEDR;  // Offset: 0x08 (Velocidad de salida)
7      volatile uint32_t PUPDR;    // Offset: 0x0C (Resistencias Pull-up/Pull-down)
8      volatile uint32_t IDR;      // Offset: 0x10 (Registro de entrada)
9      volatile uint32_t ODR;      // Offset: 0x14 (Registro de salida)
10     uint32_t RESERVED[2];        // Offset: 0x18 - 0x1F (Hueco reservado en
                                   hardware)
11     volatile uint32_t BSRR;      // Offset: 0x20 (Bit Set/Reset Register)
12 } GPIO_TypeDef;
13
14 // Direccion base del periférico GPIOA
15 #define GPIOA_BASE (0x40020000U)
16
17 // Creación de la variable para manejar el periférico
18 #define GPIOA ((GPIO_TypeDef *) GPIOA_BASE)
19
20 void ejemplo_estructura(void) {
21     // Acceso directo a cualquier registro del periférico
22     GPIOA->ODR |= (1U << 7);    // Pone a '1' el Pin 7
23 }

```

Alineación de memoria: Es imprescindible respetar los huecos de direcciones no utilizadas mediante miembros de relleno (`RESERVED`), garantizando que cada miembro de la estructura coincida exactamente con la dirección del registro físico correspondiente. También hay que tener cuidado si se utilizan miembros de tipos cortos como `uint16_t` o `uint8_t`, ya que el compilador podría introducir huecos por su propia cuenta para que variables completas de tipo `uint32_t` estén alineadas a direcciones múltiplo de cuatro. Normalmente esto no es un problema, pero conviene tenerlo en cuenta.

1.3. Utilización de `union` y campos de bits

Aunque las operaciones bit a bit con máscaras (`&`, `—`) son el estándar industrial, C permite mapear campos de bits (*bit-fields*) mediante `struct` dentro de una `union`. Esto habilita el acceso individual a un bit o grupo de bits sin requerir desplazamientos explícitos.

```

1  typedef union {
2      uint32_t raw; // Acceso al registro completo de 32 bits
3
4      struct {
5          uint32_t ENABLE : 1; // Bit 0: Activación del periférico
6          uint32_t MODE   : 2; // Bits [2:1]: Función (00, 01, 10, 11)
7          uint32_t INT_EN  : 1; // Bit 3: Habilitar interrupción
8          uint32_t RESERVED : 28; // Bits [31:4]: No utilizados
9      } bits;
10 } CR_Register_t;
11
12 void configurar_registro(void) {
13     volatile CR_Register_t *CR = (CR_Register_t *) 0x40002000U;
14
15     // Escritura directa en campos concretos sin alterar los demás
16     CR->bits.ENABLE = 1;
17     CR->bits.MODE   = 0b01;
18
19     // 0 bien modificación del registro completo en una sola operación

```

```

20     CR->raw = 0x0000000B;
21 }

```

Advertencia sobre portabilidad: La especificación del estándar C deja a criterio de la implementación del compilador la ordenación de los campos de bits (de LSB a MSB o viceversa) y el empaquetado en memoria. Además, modificar un campo de bits genera internamente una secuencia **Read-Modify-Write** no atómica, lo que puede derivar en condiciones de carrera si el registro es accedido simultáneamente por una ISR.

1.4. Registros mayores al tamaño de palabra

Generalmente, los registros de los periféricos serán de 32 bits, con diferentes funciones para cada bit que podremos configurar, de manera sencilla, activando o desactivando dichos bits. En ocasiones, es posible encontrar periféricos con registros donde el tamaño es mayor al de palabra, y por tanto para manejar su funcionamiento, debemos movernos no sólo en los bits de un registro, sino a través de varios registros. Considera por ejemplo el siguiente código:

```

1 typedef struct {
2     volatile uint32_t ISER[8];
3     ...
4     volatile uint32_t ICER[8];
5     ...
6 } NVIC;

```

En este caso, el registro `ISER` o `ICER` tienen 256 bits de ancho (para gestionar, a través de cada uno, las diferentes fuentes de interrupción del sistema). Si queremos modificar la fuente 71, ¿cómo lo hacemos?

Si miramos los registros, veremos que en `ISER[0]` están los primeros 32 bits (0-31), en `ISER[1]` los siguientes (32-63), en `ISER[2]` los siguientes (64-95) y así sucesivamente. Por ejemplo, el bit 71 de `ISER` será, de manera equivalente, el bit 7 de `ISER[2]`. Dicho de otra manera, el resultado de **dividir** el bit entre el número de bits por sub-registro nos dará el índice de sub-registro, y el resto de dicha división será la posición a modificar dentro de ese registro. Podemos verlo en la Figura 1.

Tanto la división como el resto, al ser generalmente el divisor una potencia de dos, se pueden hacer con simples desplazamientos y máscaras bit a bit.

También cabe la posibilidad de que estos registros múltiples no tengan necesariamente un único bit por función. En estos casos, deberemos de igual manera calcular, en primer lugar, el sub-registro correspondiente mediante una división y, en un segundo lugar, el rango de bits dentro de ese sub-registro que queremos modificar mediante un módulo.

1.5. Palabra clave `static`

La palabra clave `static` tiene dos usos principales en C:

1. Hacer que una variable, dentro de una función, retenga el valor entre llamadas. Aunque no le daremos este uso durante el curso, conviene saberlo.

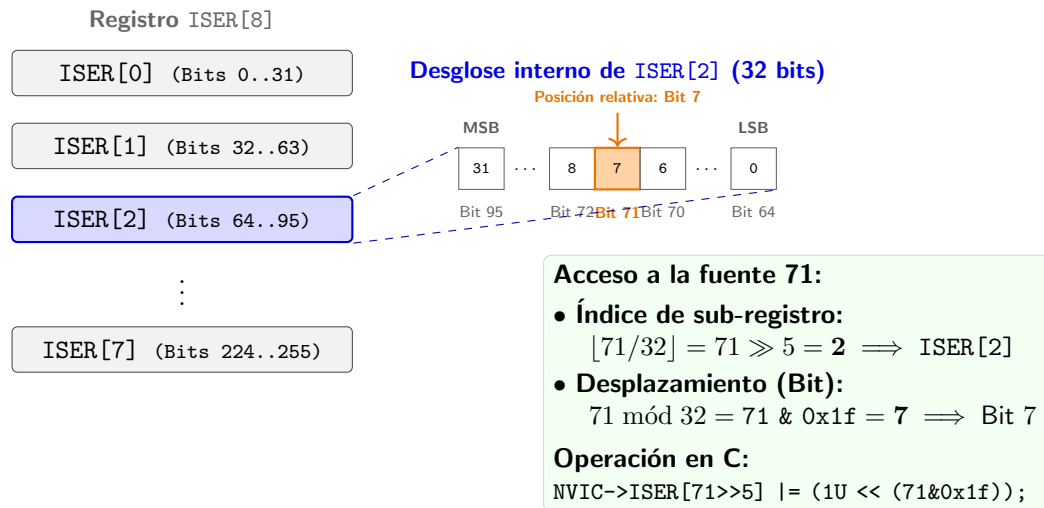


Figura 1: Esquema de direccionamiento de interrupciones en el registro ISER[8].

2. Hacer que una variable o función sólo sea vista dentro de la unidad de compilación a la que pertenece. Así, esas variables o funciones no pueden ser vistas por otras unidades de compilación (normalmente archivos .c), y evitamos colisiones de nombres.

1.5.1. Funciones `static inline`

A la hora de realizar operaciones frecuentemente repetidas (como por ejemplo activar o desactivar un pin de un periférico de GPIO), la aproximación tradicional ha sido la de definir macros parametrizadas mediante `#define`. De esta forma, se evita la sobrecarga de llamar a una función específica, consistente en salvar registros en la pila, además de realizar el salto y retorno.

```
1 #define TOGGLE_PIN_MACRO(port, pin) ((port)->ODR ^= (1U << (pin)))
2
3 // Evaluacion con incremento: TOGGLE_PIN_MACRO(GPIOA, i)
4 // expande a: (GPIOA)->ODR ^= (1U << (i))
```

Las macros, sin embargo, presentan algunos inconvenientes, especialmente cuando empieza a haber muchas: La trazabilidad se hace complicada, tanto en el desarrollo como en la depuración. Adicionalmente, los errores de sintaxis se hacen más comunes debido a las reglas de sustitución del preprocesador.

Como alternativa, disponemos de las funciones `static inline`. La palabra clave `inline` hace que el compilador reemplace la llamada a la función directamente por su código fuente en el punto de llamada. Por su parte, `static`, como ya decíamos, hace que la función no sea vista fuera de la unidad de compilación, así que aunque la incluyamos desde numerosos archivos .c, evitaremos errores de duplicidad al compilar.

```
1 #include <stdint.h>
2
3 static inline void gpio_toggle_pin(GPIO_TypeDef *port, uint8_t pin) {
4     // Comprobación de tipos en compilación y sin efectos secundarios
5     port->ODR ^= (1U << pin);
6 }
```

Así, se consigue la rapidez de las macros, añadiendo las comprobaciones extra del compilador

y funciones de depuración estándar.